ICT Engineering for Smart Societies

Course: Advanced Machine Learning for Image Processing

Laboratory 1 Report

Academic Year 2025/2026

Students:

Lorenzo Cavallaro - s346742
Emanuele Cumino - s361790

Contents

1 Exercise 1 - Image Denoising **3**
1.1 Introduction . . . 3
1.2 Methodology . . . 3
1.2.1 Dataset . . . 3
1.2.2 Network Architecture . . . 3
1.2.3 Training Protocol . . . 4
1.2.4 Evaluation Metrics . . . 4
1.3 Experiments & Results . . . 5
1.3.1 Residual vs Non-Residual Comparison ($\sigma = 25$) Training . . . 5
1.3.2 Generalization to Out-of-Distribution Noise ($\sigma = 10, 50$) . . . 5
1.3.3 Blind Denoising (Variable Noise Training $\sigma \in [10, 50]$) . . . 7
1.4 Analysis . . . 8
1.4.1 Residual Learning Impact . . . 8
1.4.2 Out-of-Distribution Generalization . . . 8
1.5 Conclusion . . . 9

2 Exercise 2 - Satellite Image Super-Resolution **10**
2.1 Introduction . . . 10
2.2 Dataset . . . 10
2.3 DataLoader . . . 10
2.4 Model . . . 10
2.5 Training . . . 11
2.6 Results . . . 11
2.6.1 Model trained with 50 epochs . . . 11
2.6.2 Model trained with 1000 epochs . . . 12
2.7 Conclusion . . . 12

Abstract

This report presents the implementation and evaluation of two convolutional neural network approaches for image restoration tasks: grayscale image denoising and satellite image super-resolution. In the first part, a DnCNN model is trained to remove additive white Gaussian noise using a patch-based strategy. Different architectural and training design choices are investigated, including residual versus non-residual learning, normalization methods, and robustness to mismatched noise levels. Results show that residual learning significantly improves performance, achieving up to 28.88 dB PSNR at the target noise level, while blind training enhances robustness across varying noise conditions at the cost of peak performance. In the second part, a super-resolution model inspired by RCAN is implemented to upscale satellite images by a factor of four. The network leverages residual channel attention blocks and is trained using an L1 loss on cropped image patches. Experimental results demonstrate that increasing training epochs leads to moderate improvements, reaching an average PSNR of 30.23 dB. Overall, the study highlights the importance of architectural choices and training strategies in balancing performance and generalization. While specialized models achieve higher accuracy under fixed conditions, more flexible approaches provide better robustness in realistic scenarios.

1 Exercise 1 - Image Denoising

1.1 Introduction

In this exercise, we implemented, trained and evaluated a shortened DnCNN for grayscale image denoising under additive white Gaussian noise (AWGN). The dataset contains 443 images, split into 431 training images and 12 test images. Training is patch-based: a random 32×32 crop is extracted from each selected training image, then synthetic AWGN is added.

The objective is to reconstruct the clean image from a noisy input and evaluate performance with PSNR. Besides the baseline setting with fixed noise level $\sigma = 25$, we also studied:

- residual vs non-residual architectures,
- robustness to mismatched test noise ($\sigma = 10$ and $\sigma = 50$),
- blind denoising with random training noise $\sigma \sim \mathcal{U}(10, 50)$.

1.2 Methodology

1.2.1 Dataset

Each image is converted to grayscale and normalized to $[0, 1]$. For training, a random spatial crop of size 32×32 is sampled at every access, so the model sees different local contexts across epochs.

Given clean patch x (in $[0, 255]$ scale before normalization), noisy patch y is generated as:

$$y = x + n, \quad n \sim \mathcal{N}(0, \sigma^2),\ \sigma = 25.$$

After clipping to valid intensity range, both clean and noisy patches are normalized to $[0, 1]$ and converted to `float32` tensors.

For evaluation on the test set, full-resolution images are used (no patch split), exploiting the fully-convolutional nature of the network.

1.2.2 Network Architecture

We implemented a DnCNN with:

- input convolution: $1 \to C$ channels,
- 8 repeated Conv-BN-ReLU blocks,
- output convolution: $C \to 1$ channel.

We tested different channel widths and selected $C = 64$ as the best trade-off between representation power and generalization. A $C = 128$ variant also achieved high training quality, but showed slightly stronger overfitting in validation/test behavior.

Residual formulation analysis:

- tested forms: $\hat{x} = x - f(x)$, $\hat{x} = x + f(x)$, and direct noise prediction with loss on noise,
- selected form: $\hat{x} = x - f(x)$.

This choice is mathematically consistent with denoising residual learning. If $y = x + n$, then predicting residual correction is equivalent (up to sign convention) to predicting noise.

1.2.3 Training Protocol

Optimization setup. All models are trained with Adam optimizer and MSE loss. We used GPU acceleration when available.

Validation-driven training. Instead of fixing short training horizons (e.g., 75 or 100 epochs), we moved to a validation-driven protocol with:

- explicit train/validation split from training set,
- PSNR monitoring on validation data each epoch,
- learning-rate scheduler `ReduceLROnPlateau` in `mode=max` on validation PSNR,
- total training length of 200 epochs.

This setup is important because denoising quality is judged in PSNR (not only training loss). The scheduler progressively reduces LR when PSNR saturates, enabling later-stage refinement.

Batch size and normalization ablations. We selected batch size 16 after empirical tests showing lower test PSNR with larger batches. This is coherent with common generalization behavior:

- larger batches reduce gradient stochasticity,
- reduced stochasticity may bias optimization toward sharper minima,
- for fixed epochs, larger batch size also means fewer parameter updates.

We also tested GroupNorm (8, 16, 32 groups) as replacement for BatchNorm. GroupNorm variants gave very high training PSNR but lower test PSNR, indicating stronger overfitting under this specific data regime and model size. BatchNorm provided the best train/test balance in our experiments.

1.2.4 Evaluation Metrics

Primary metric is PSNR:

$$\mathrm{PSNR}_{\mathrm{dB}} = 10 \log_{10} \left(\frac{1}{\mathrm{MSE}} \right)$$

since images are normalized to $[0, 1]$.

We report the following averages over all test images:

- noisy-vs-clean PSNR,
- denoised-vs-clean PSNR

Table 1: Fixed-noise test results ($\sigma = 25$).

Metric	Residual model	Non-residual model
Test loss (MSE)	0.0013	0.0016
Avg noisy PSNR (dB)	20.33	20.33
Avg denoised PSNR (dB)	28.88	28.12
Best validation PSNR (dB)	30.10	29.33

1.3 Experiments & Results

1.3.1 Residual vs Non-Residual Comparison ($\sigma = 25$) Training

Both models were trained with the same data split and protocol to ensure fair comparison.

Residual learning gives a consistent gain in both validation and test PSNR.

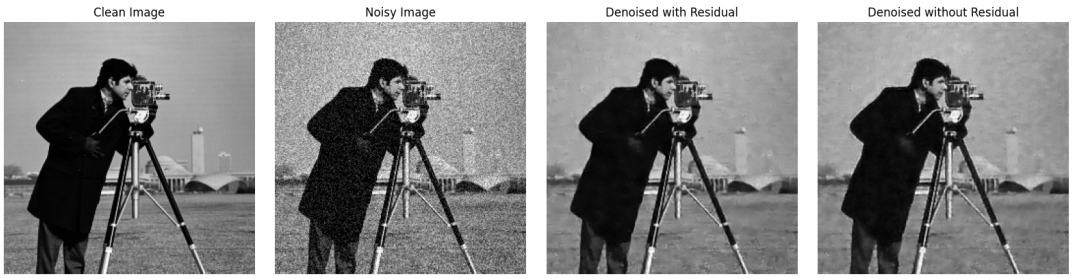


Figure 1: Qualitative comparison between residual and non-residual denoising outputs on the same test image.

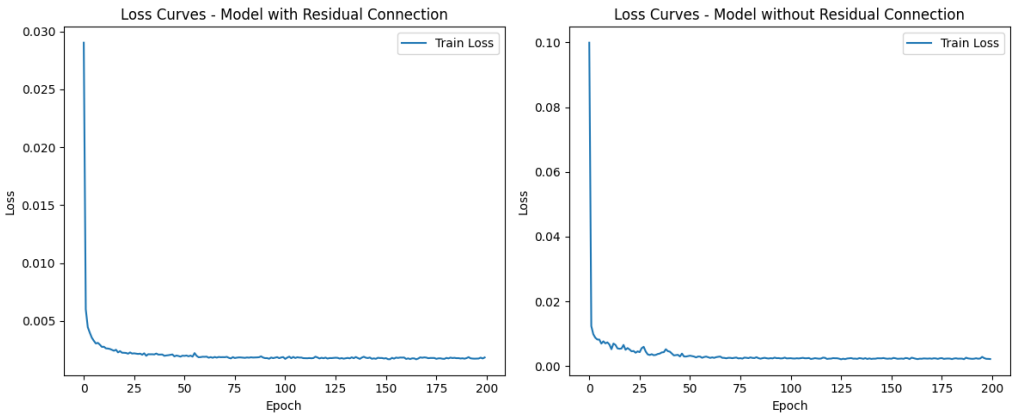


Figure 2: Training loss comparison for residual and non-residual models.

1.3.2 Generalization to Out-of-Distribution Noise ($\sigma = 10, 50$)

Using the model trained at fixed $\sigma = 25$, we tested denoising on mismatched noise levels.

As expected, denoising gain is present at both levels, but quality drops significantly at $\sigma = 50$ because this regime is far from the training condition.

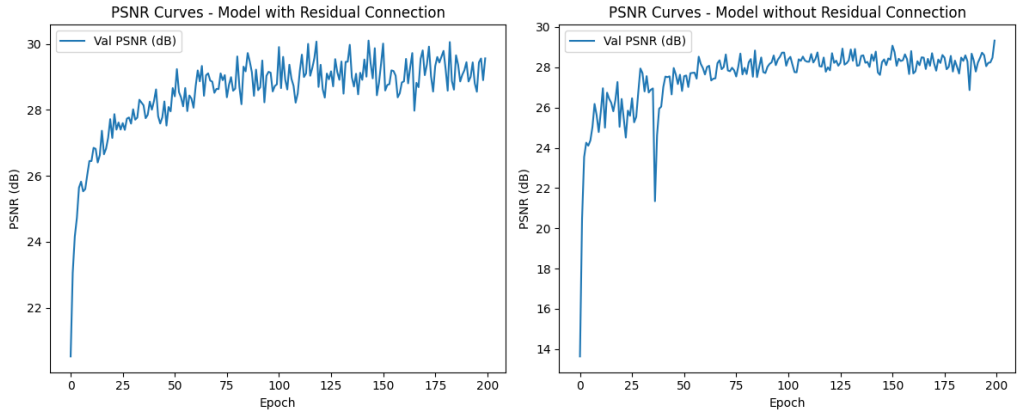


Figure 3: Validation PSNR trends across epochs for both architectures.

Table 2: Generalization of fixed-σ model to out-of-distribution noise.

Noise std	Noisy PSNR (dB)	Denoised PSNR (dB)
$\sigma = 10$	28.16	30.06
$\sigma = 50$	14.75	18.22

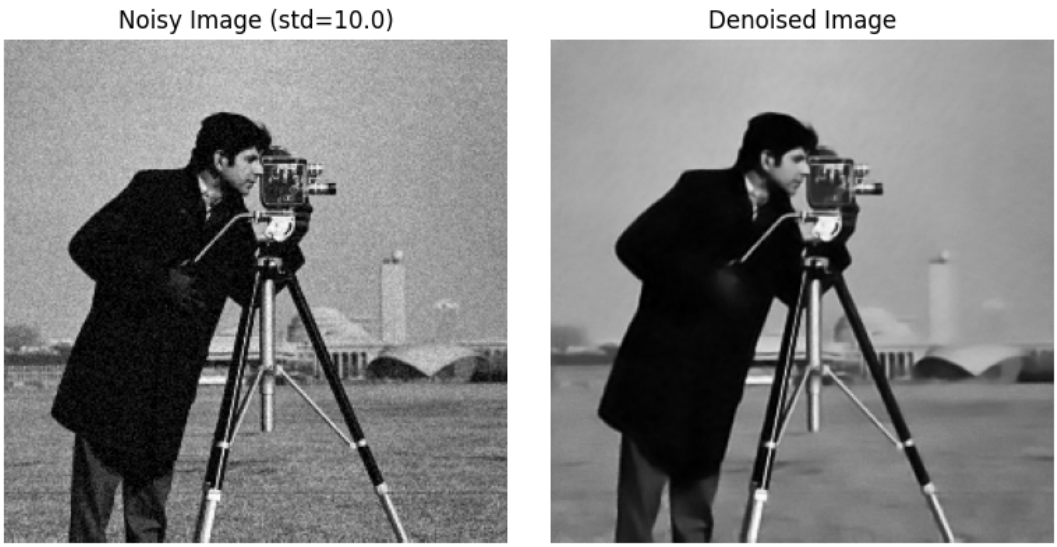


Figure 4: Qualitative behavior of fixed-$\sigma = 25$ model under mismatched noise levels of $\sigma = 10$

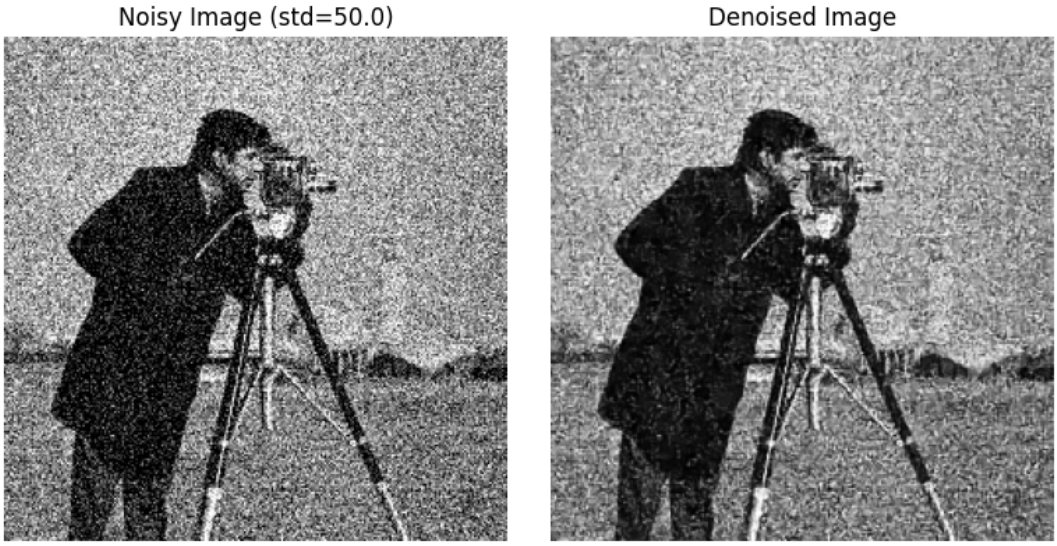


Figure 5: Qualitative behavior of fixed-$\sigma = 25$ model under mismatched noise levels of $\sigma = 50$

1.3.3 Blind Denoising (Variable Noise Training $\sigma \in [10, 50]$)

We trained a blind model where each sample uses random $\sigma \sim \mathcal{U}(10, 50)$ during data generation.

Table 3: Blind model performance across noise levels.

Noise std	Noisy PSNR (dB)	Denoised PSNR (dB)
$\sigma = 10$	28.16	31.63
$\sigma = 25$	20.34	27.79
$\sigma = 50$	14.77	23.88

Compared to the fixed-$\sigma = 25$ model, blind training strongly improves robustness at high noise (notably at $\sigma = 50$), while showing lower performance around the nominal $\sigma = 25$ point.

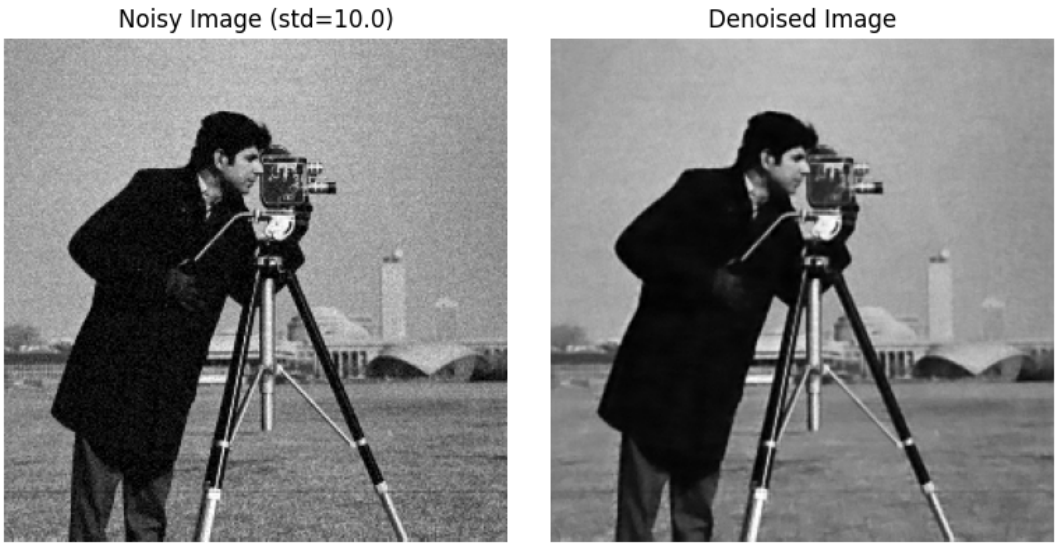


Figure 6: Blind denoiser qualitative examples at $\sigma = 10$ noise level.

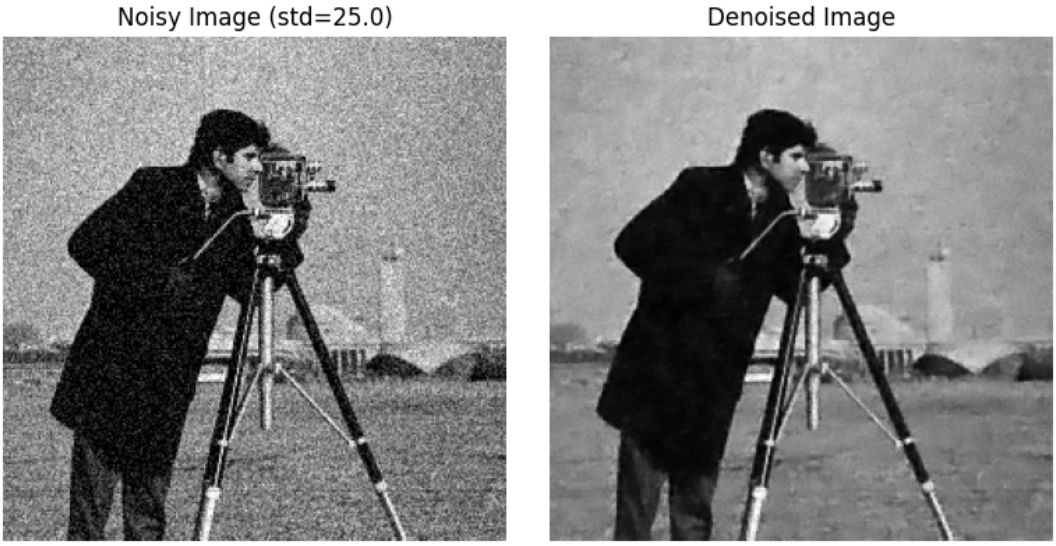


Figure 7: Blind denoiser qualitative examples at $\sigma = 25$ noise level.

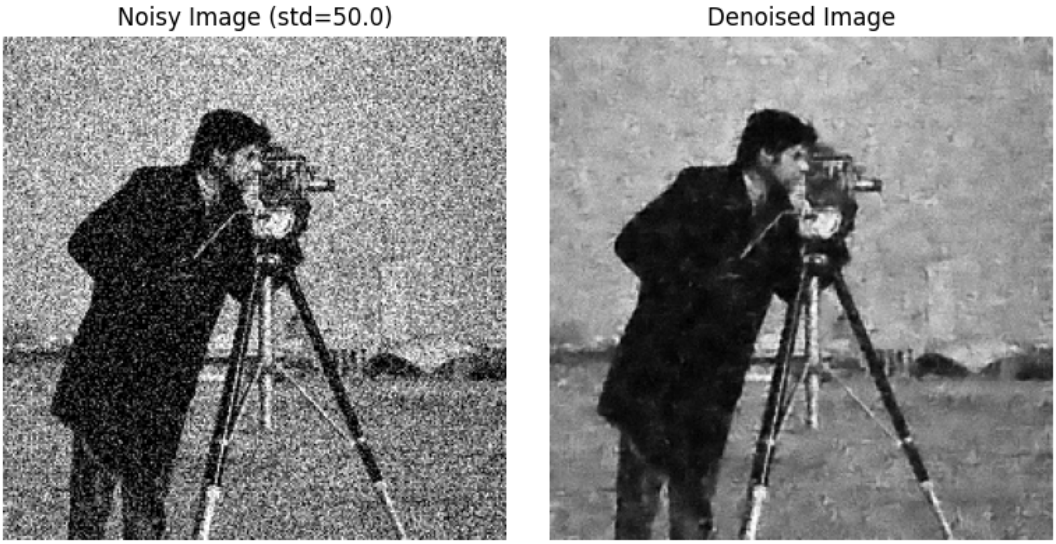


Figure 8: Blind denoiser qualitative examples at $\sigma = 50$ noise level.

1.4 Analysis

1.4.1 Residual Learning Impact

Residual learning is the key contributor to stable optimization and better reconstruction quality in this setup. Relative to direct clean-image prediction, the residual model improved test PSNR by 0.76 dB at $\sigma = 25$ (Table 1).

The reason is that predicting residual content focuses model capacity on high-frequency perturbations/noise components instead of re-synthesizing the full image structure from scratch.

1.4.2 Out-of-Distribution Generalization

The fixed-σ model generalizes reasonably to nearby noise levels but degrades for large mismatch ($\sigma = 50$), as shown in Table 2. This behavior is expected: the learned denoising prior is specialized around the training noise distribution.

Blind training trades specialization for robustness. It is not optimized for a single noise level, therefore peak PSNR at $\sigma = 25$ can be lower, but it is significantly more reliable across heterogeneous noise conditions, which is preferable in practical settings where noise level is unknown (Table 3).

1.5 Conclusion

This exercise demonstrates a complete denoising workflow from data simulation to architecture design and robust evaluation. The final design choices were guided by systematic ablations:

- batch size 16 for better generalization,
- BatchNorm over GroupNorm to reduce overfitting,
- 64 channels as best complexity/generalization trade-off,
- validation-PSNR-driven long training with LR scheduling,
- residual formulation selected after analytical and empirical comparison.

Overall, results show strong denoising performance at the target noise level and clear evidence-based understanding of the trade-off between specialization and robustness.

2 Exercise 2 - Satellite Image Super-Resolution

2.1 Introduction

The following exercise requests to create a neural network in order to super-resolve satellite images by a factor of 4. It provides a dataset of RGB images with 400 training scenes and 20 testing scenes. The aim is to provide super-resolve images from low resolution ones. The following paragraphs will describe the necessary steps to implement the neural network.

2.2 Dataset

First of all, we defined a Pytorch dataset; the class SuperResolutionDataset takes as input the path where the training images are stored, both low and high resolution images. Here, the aim is to crop a small area of a low resolution image and then crop the relative area of the high resolution one. In this way we obtained a dataset that occupies less space than the initial one. The starting point of the cropping area is chosen randomly. The class returns the cropping images as a tuple with a float32 tensor for the low-resolution image patch and another float32 tensor for the high-resolution image patch.

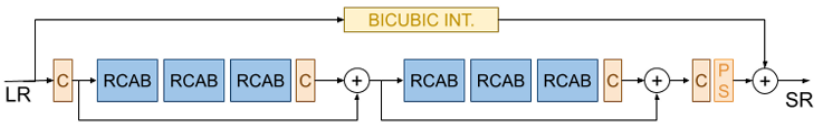
2.3 DataLoader

We defined the Dataloader respectively for the training and the test. In the training dataloader we decided batch size 16, which we see is a reasonable value; we put the shuffle variable to True and we set the number of process running in parallel to 0, because we saw that this has led to crashes. In the test one we just took one input at a time and we don't want the data to be shuffled.

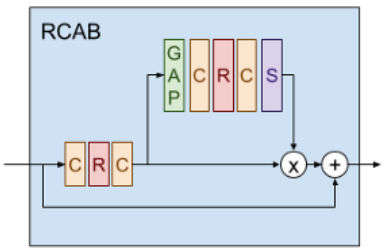
2.4 Model

Therefore, we implemented the network model with a CNN that is similar to a RCAN one, used to create super-resolution images thanks to the residual learning and the channel attention concept.

We created the model proposed in the lab description:



Where the RCAB blocks are structured in the following way:



The "C" stands for the convolution block, the "R" stands for the ReLU one, the "GAP" stands for the Global Average Pool, the "S" for Sigmoid and "PS" for pixel shuffle.

2.5 Training

Therefore, we can finally train the model with a L1 loss, which is a way to measure the error as mean absolute error.
We decided to train the model with 50 epochs and 100 epochs and then compare their average loss:

- model trained with 50 epochs: 0.019856
- model trained with 100 epochs: 0.019739

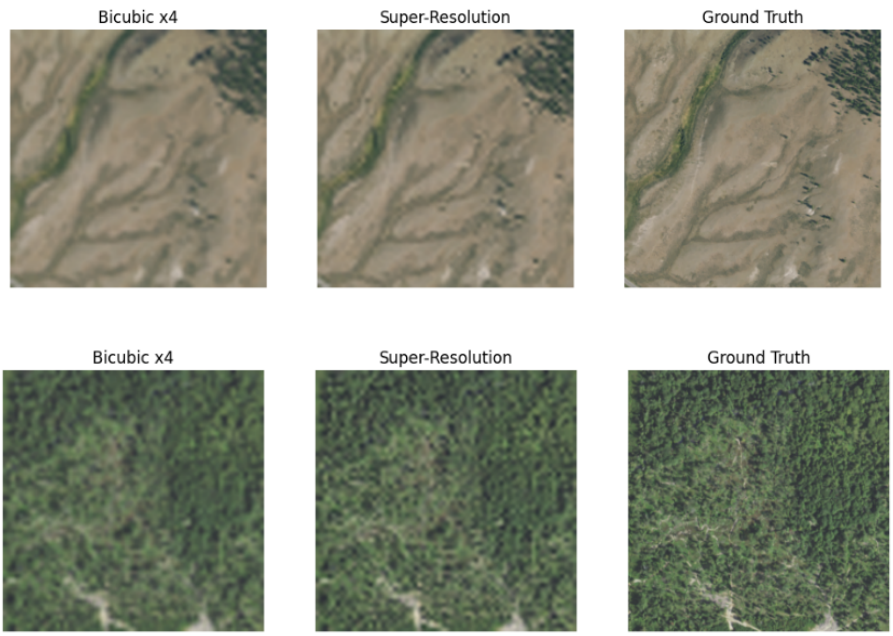
Increasing the epochs, we noticed that the average loss didn't decrease linearly, sometimes the value

2.6 Results

After the training session, we can set the model to evaluation mode and we can compare the bigger original image, the super-resolve ones and the high-resolution ones. As a comparison criteria, we choose the PSNR (Peak signal-to-noise ratio) to see how good were the results.

2.6.1 Model trained with 50 epochs

Consider two images and see how their PSNR values are:



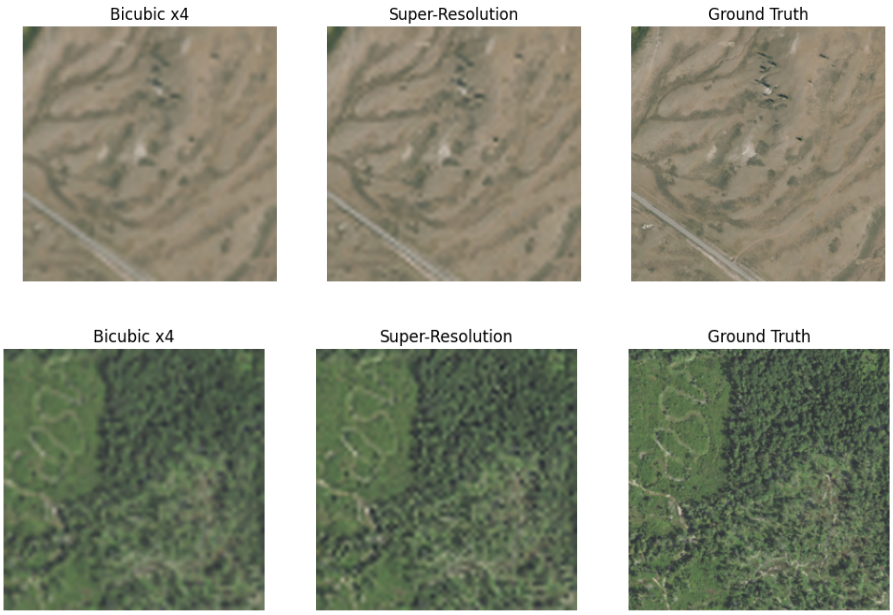
- Image 1: 31.57 dB
- Image 2: 26.46 dB

The average PSNR is:

29.8579 dB

2.6.2 Model trained with 1000 epochs

Consider two images and see how their PSNR values are:



- Image 1: 34.39 dB
- Image 2: 27.27 dB

The average PSNR is:

30.2302 dB

2.7 Conclusion

By comparing the results from the two different models trained, we noticed that the average loss and the PSNR values are slightly different: the 100 epochs model has better results, even though sometimes this difference is not so significant.
We were able to reach good results (PSNR = 30.23 db); however, the super-resolute images are clearly not perfect as the ground truth.
For a better estimation we may use other parameters like SSIM (Structural Similarity Index), which are more significant for the way we human perceive images.